```
"""
Rasengan Hand Tracker
---------------------
Detects an open palm using your webcam and draws a glowing chakra-style
orb above it in real time, using OpenCV for video/graphics and
MediaPipe for hand landmark detection.

Author: Trusha Y
Built with: Python, OpenCV, MediaPipe

How it works:
1. MediaPipe Hands finds 21 landmark points per detected hand.
2. We check if the palm is "open" by comparing each fingertip's
   distance from the wrist to its knuckle's distance from the wrist.
   If a fingertip is farther out than its knuckle, that finger is
   extended.
3. If 3 or more fingers are extended, we treat the palm as open and
   grow a glowing circle ("orb") above the palm. If the palm closes,
   the orb fades out.
4. The orb is drawn with several soft, semi-transparent circles of
   decreasing radius to fake a glow, and
```

```
rotating "swirl" lines to
   give it some motion, similar to a chakra
spiral.

Run:
    pip install -r requirements.txt
    python rasengan.py

Press 'q' to quit.
"""

import cv2
import math
import time
import numpy as np
import mediapipe as mp

# ----------------------------------------
---------------------------------
# Setup
# ----------------------------------------
---------------------------------

mp_hands = mp.solutions.hands
hands = mp_hands.Hands(
    max_num_hands=2,
    min_detection_confidence=0.65,
    min_tracking_confidence=0.65,
)
mp_draw = mp.solutions.drawing_utils

# Landmark indices for fingertips and their
corresponding knuckle (PIP) joints
FINGERTIPS = [4, 8, 12, 16, 20]   # thumb,
```

```
index, middle, ring, pinky tips
FINGER_PIPS = [3, 6, 10, 14, 18]  #
matching lower joints
WRIST = 0

# Per-hand "charge" state (0.0 = no orb,
1.0 = fully formed orb)
hand_charge = {"Left": 0.0, "Right": 0.0}

CHARGE_UP_SPEED = 0.06     # how fast the
orb grows when palm is open
CHARGE_DOWN_SPEED = 0.18   # how fast the
orb fades when palm closes

# Colors (BGR) per hand, so two hands look
visually distinct
PALETTES = {
    "Right": {"core": (255, 255, 255),
"glow": (0, 140, 255)},   # warm orange
    "Left":  {"core": (255, 255, 255),
"glow": (255, 200, 0)},   # cool cyan/blue
}



# ----------------------------------------
---------------------------------
# Helper functions
# ----------------------------------------
---------------------------------

def distance(a, b):
    """Euclidean distance between two (x,
y) points."""
    return math.hypot(a[0] - b[0], a[1] -
```

```
b[1])


def is_palm_open(landmarks, w, h):
    """
    Returns True if 3 or more fingers are
extended.
    landmarks: list of 21 (x, y) pixel-
coordinate points for one hand.
    """
    wrist = landmarks[WRIST]
    extended = 0
    for tip_idx, pip_idx in zip(FINGERTIPS,
FINGER_PIPS):
        tip = landmarks[tip_idx]
        pip = landmarks[pip_idx]
        if distance(tip, wrist) >
distance(pip, wrist):
            extended += 1
    return extended >= 3


def hand_size(landmarks):
    """Rough hand size estimate, used to
scale the orb to the hand."""
    wrist = landmarks[WRIST]
    middle_tip = landmarks[12]
    return distance(wrist, middle_tip)


def palm_center(landmarks):
    """Average of wrist + finger base
knuckles = approximate palm center."""
    base_points = [landmarks[i] for i in
```

```
[0, 5, 9, 13, 17]]
    xs = [p[0] for p in base_points]
    ys = [p[1] for p in base_points]
    return (sum(xs) / len(xs), sum(ys) /
len(ys))


def draw_orb(frame, center, radius, charge,
colors, t):
    """
    Draws a glowing, swirling orb centered
above the palm.
    Uses an overlay + alpha blending so the
glow looks soft instead
    of a hard-edged circle.
    """
    if radius <= 2 or charge <= 0.02:
        return

    overlay = frame.copy()
    cx, cy = int(center[0]), int(center[1])

    # Outer glow rings (large, faint, glow
color)
    for i in range(4, 0, -1):
        r = int(radius * (1 + i * 0.18))
        alpha = 0.08 * charge
        cv2.circle(overlay, (cx, cy), r,
colors["glow"], -1)
        cv2.addWeighted(overlay, alpha,
frame, 1 - alpha, 0, frame)
        overlay = frame.copy()

    # Mid glow (medium, glow color, more
```

```
opaque)
    cv2.circle(overlay, (cx, cy),
int(radius * 1.1), colors["glow"], -1)
    cv2.addWeighted(overlay, 0.35 * charge,
frame, 1 - 0.35 * charge, 0, frame)
    overlay = frame.copy()

    # Bright core (small, near-white, fully
opaque at full charge)
    cv2.circle(overlay, (cx, cy),
int(radius * 0.55), colors["core"], -1)
    cv2.addWeighted(overlay, 0.9 * charge,
frame, 1 - 0.9 * charge, 0, frame)

    # Rotating swirl lines for a "spinning
chakra" feel
    for k in range(3):
        angle = t * 4 + k * (2 * math.pi /
3)
        x2 = int(cx + math.cos(angle) *
radius * 0.9)
        y2 = int(cy + math.sin(angle) *
radius * 0.9)
        cv2.line(frame, (cx, cy), (x2, y2),
colors["glow"], 2, cv2.LINE_AA)


# ----------------------------------------
---------------------------------
# Main loop
# ----------------------------------------
---------------------------------

def main():
```

```
    cap = cv2.VideoCapture(0)
    if not cap.isOpened():
        print("Could not open webcam.")
        return

    start_time = time.time()

    while True:
        ok, frame = cap.read()
        if not ok:
            break

        frame = cv2.flip(frame, 1)  #
mirror for natural interaction
        h, w, _ = frame.shape
        rgb = cv2.cvtColor(frame,
cv2.COLOR_BGR2RGB)
        results = hands.process(rgb)

        t = time.time() - start_time

        if results.multi_hand_landmarks and
results.multi_handedness:
            for hand_landmarks, handedness
in zip(

results.multi_hand_landmarks,
results.multi_handedness
            ):
                label =
handedness.classification[0].label  #
"Left" or "Right"

                # Convert normalized
```

```
landmarks to pixel coordinates
                pts = [(lm.x * w, lm.y * h)
for lm in hand_landmarks.landmark]

                # Draw the hand skeleton
                mp_draw.draw_landmarks(
                    frame, hand_landmarks,
mp_hands.HAND_CONNECTIONS
                )

                # Update charge based on
whether palm is open
                open_now =
is_palm_open(pts, w, h)
                if open_now:
                    hand_charge[label] =
min(1.0, hand_charge[label] +
CHARGE_UP_SPEED)
                else:
                    hand_charge[label] =
max(0.0, hand_charge[label] -
CHARGE_DOWN_SPEED)

                charge = hand_charge[label]
                if charge > 0:
                    size = hand_size(pts)
                    center =
palm_center(pts)
                    # Lift the orb above
the palm
                    orb_center =
(center[0], center[1] - size * 1.3)
                    radius = size * 0.9 *
charge
```

```
                    colors =
PALETTES.get(label, PALETTES["Right"])
                    draw_orb(frame,
orb_center, radius, charge, colors, t)
        else:
            # No hands visible this frame -
> let any existing orb fade out
            for label in hand_charge:
                hand_charge[label] =
max(0.0, hand_charge[label] -
CHARGE_DOWN_SPEED)

        cv2.putText(
            frame, "Open your palm to
charge an orb | 'q' to quit",
            (10, h - 15),
cv2.FONT_HERSHEY_SIMPLEX, 0.55, (255, 255,
255), 1, cv2.LINE_AA
        )
        cv2.imshow("Chakra Orb Tracker",
frame)

        if cv2.waitKey(1) & 0xFF ==
ord('q'):
            break

    cap.release()
    cv2.destroyAllWindows()


if __name__ == "__main__":
    main()
```